

BABU BANARASI DAS UNIVERSITY

SCHOOL OF COMPUTER APPLICATIONS

NoSQL PROJECT ON

EMPLOYEE MANAGEMENT SYSTEM

USING MONGODB

Submitted To	Mr. Harendra Singh
Submitted By	Arpit Srivastava
Course	BCA (DS & AI)
University Roll No.	1250258130

Academic Project Report

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teacher, Mr. Harendra Singh, for providing guidance and support during the development of this NoSQL project. I am also thankful to my university and the School of Computer Applications for providing the learning environment and resources required to complete this work.

This project helped me understand the practical use of MongoDB as a NoSQL database and gave me hands-on experience with database creation, collections, CRUD operations, query operators, logical operators, sorting, limiting, searching and counting documents.

1. INTRODUCTION

Employee Management System is a database-based application concept used to store and manage employee information in an organized manner. In this project, MongoDB is used as the NoSQL database to store employee documents.

MongoDB stores data in flexible document-oriented structures. Each employee document contains fields such as employee ID, name, age, salary, city and department. The project demonstrates how MongoDB can be used to insert, retrieve, update, delete, filter, sort and count employee records.

2. OBJECTIVES

- To understand the basic concepts of NoSQL databases.
- To create and use a MongoDB database and collection.
- To insert and manage employee records.
- To perform CRUD operations using MongoDB queries.
- To understand comparison operators such as \$eq, \$gt, \$lt, \$gte and \$lte.
- To use logical operators such as \$and and \$or.
- To perform sorting, limiting, searching and counting operations.
- To gain practical experience with MongoDB and mongosh.

3. DATABASE AND COLLECTION

Database used in the working session:

```
use EmployeeDB
```

Collection:

```
db.createCollection("employees")
```

The employee collection is designed with the following fields:

Field	Description
employeeid	Unique employee identifier
name	Employee name
age	Employee age
salary	Employee salary
city	Employee city
department	Employee department

4. MONGODB OPERATIONS

The following sections document the MongoDB operations performed for the Employee Management System. Screenshots can be placed below each query. Screenshots available from the completed working session have been included near the end of this report.

4.1 CREATE DATABASE

MongoDB Query:

```
use EmployeeDB
```

Explanation: Selects/creates the EmployeeDB database.

SCREENSHOT PLACEHOLDER

4.1 CREATE DATABASE result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.2 CREATE COLLECTION

MongoDB Query:

```
db.createCollection("employees")
```

Explanation: Creates a collection named employees.

SCREENSHOT PLACEHOLDER

4.2 CREATE COLLECTION result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.3 INSERT 50 RECORDS

MongoDB Query:

```
db.employees.insertMany([
  {employeeId:1, name:"...", age:35, salary:85000, city:"New York",
  department:"Technology"},
  {employeeId:2, name:"...", age:34, salary:82000, city:"Brooklyn",
  department:"Security"},
  // remaining employee documents through employeeId:50
])
```

Explanation: Inserts the project's 50 employee documents into the employees collection.

SCREENSHOT PLACEHOLDER

4.3 INSERT 50 RECORDS result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.4 DISPLAY ALL RECORDS

MongoDB Query:

```
db.employees.find()
```

Explanation: Retrieves employee documents from the collection.

*SCREENSHOT PLACEHOLDER
4.4 DISPLAY ALL RECORDS result*

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.5 \$EQ — EQUAL TO

MongoDB Query:

```
db.employees.find({salary:{$eq:60000}})
```

Explanation: Finds employees whose salary is exactly 60000.

*SCREENSHOT PLACEHOLDER
4.5 \$EQ — EQUAL TO result*

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.6 \$GT — GREATER THAN

MongoDB Query:

```
db.employees.find({salary:{$gt:70000}})
```

Explanation: Finds employees whose salary is greater than 70000.

*SCREENSHOT PLACEHOLDER
4.6 \$GT — GREATER THAN result*

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.7 \$LT — LESS THAN

MongoDB Query:

```
db.employees.find({salary:{$lt:50000}})
```

Explanation: Finds employees whose salary is less than 50000.

SCREENSHOT PLACEHOLDER

4.7 \$LT — LESS THAN result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.8 \$GTE — GREATER THAN OR EQUAL

MongoDB Query:

```
db.employees.find({salary:{$gte:80000}})
```

Explanation: Finds employees whose salary is 80000 or above.

SCREENSHOT PLACEHOLDER

4.8 \$GTE — GREATER THAN OR EQUAL result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.9 \$LTE — LESS THAN OR EQUAL

MongoDB Query:

```
db.employees.find({salary:{$lte:55000}})
```

Explanation: Finds employees whose salary is 55000 or below.

SCREENSHOT PLACEHOLDER

4.9 \$LTE — LESS THAN OR EQUAL result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.10 UPDATE OPERATION

MongoDB Query:

```
db.employees.updateOne(
  {employeeId:5},
  {$set:{salary:75000}}
)
```

Explanation: Updates the salary of employee ID 5.

SCREENSHOT PLACEHOLDER
4.10 UPDATE OPERATION result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.11 DELETE ONE

MongoDB Query:

```
db.employees.deleteOne({employeeId:10})
```

Explanation: Deletes one employee document having employeeId 10.

SCREENSHOT PLACEHOLDER
4.11 DELETE ONE result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.12 DELETE MANY

MongoDB Query:

```
db.employees.deleteMany({salary:{$lt:30000}})
```

Explanation: Deletes employee documents whose salary is below 30000.

SCREENSHOT PLACEHOLDER
4.12 DELETE MANY result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.13 AND OPERATION

MongoDB Query:

```
db.employees.find({
  $and: [
```

```
{salary:{$gt:60000}},  
{age:{$lt:30}}  
]  
})
```

Explanation: Finds employees satisfying both conditions: salary greater than 60000 and age less than 30.

SCREENSHOT PLACEHOLDER

4.13 AND OPERATION result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.14 OR OPERATION

MongoDB Query:

```
db.employees.find(  
  $or:[  
    {city:"Lucknow"},  
    {city:"Kanpur"}  
  ]  
)
```

Explanation: Finds employees whose city is Lucknow or Kanpur.

SCREENSHOT PLACEHOLDER

4.14 OR OPERATION result

Paste the corresponding MongoDB Compass / mongosh screenshot here.

4.15 SORT ASCENDING

MongoDB Query:

```
db.employees.find().sort({salary:1})
```

Explanation: Sorts employee records by salary from lowest to highest.

4.16 SORT DESCENDING

MongoDB Query:

```
db.employees.find().sort({salary:-1})
```

Explanation: Sorts employee records by salary from highest to lowest.

4.17 LIMIT

MongoDB Query:

```
db.employees.find().limit(5)
```

Explanation: Displays only the first five documents.

4.18 SEARCH BY CITY

MongoDB Query:

```
db.employees.find({city:"Lucknow"})
```

Explanation: Searches for employee documents with city set to Lucknow.

4.19 COUNT DOCUMENTS

MongoDB Query:

```
db.employees.countDocuments()
```

Explanation: Counts the documents currently present in the employees collection.

5. COMPLETED OPERATION SCREENSHOTS

The following screenshots were captured during the completed MongoDB working session.

```
>_MONGOSH
> db.employees.find().sort({salary:1})
< {
  _id: ObjectId('6abc0bb6573dc920e82eea38'),
  employeeId: 35,
  name: 'Korg',
  age: 30,
  salary: 56000,
  city: 'Sakaar',
  department: 'Operations'
}
{
  _id: ObjectId('6abc0bb6573dc920e82eea2b'),
  employeeId: 22,
  name: 'Drax',
  age: 35,
  salary: 59000,
  city: 'Earth',
  department: 'Security'
}
{
  _id: ObjectId('6abc0bb6573dc920e82eea40'),
  employeeId: 43,
  name: 'Yondu',
  age: 50,
  salary: 60000,
  city: 'Missouri',
  department: 'Operations'
}
{
```

Sort Ascending — salary: 1

>_MONGOSH

```
> db.employees.find().sort({salary:-1})
```

```
< {  
  _id: ObjectId('6abc0bb6573dc920e82eea2d'),  
  employeeId: 24,  
  name: 'Nick Fury',  
  age: 58,  
  salary: 95000,  
  city: 'New York',  
  department: 'Management'  
}  
{  
  _id: ObjectId('6abc0bb6573dc920e82eea3e'),  
  employeeId: 41,  
  name: 'Pepper Potts',  
  age: 40,  
  salary: 91000,  
  city: 'New York',  
  department: 'Management'  
}  
{  
  _id: ObjectId('6abc0bb6573dc920e82eea18'),  
  employeeId: 3,  
  name: 'Thor',  
  age: 1500,  
  salary: 90000,  
  city: 'Asgard',  
  department: 'Operations'  
}  
}
```

Sort Descending — salary: -1

> _MONGOSH

> db.employees.find().limit(5)

```
< {
  _id: ObjectId('6abc0bb6573dc920e82eea16'),
  employeeId: 1,
  name: 'Iron Man',
  age: 35,
  salary: 85000,
  city: 'New York',
  department: 'Technology'
}
{
  _id: ObjectId('6abc0bb6573dc920e82eea17'),
  employeeId: 2,
  name: 'Captain America',
  age: 34,
  salary: 82000,
  city: 'Brooklyn',
  department: 'Security'
}
{
  _id: ObjectId('6abc0bb6573dc920e82eea18'),
  employeeId: 3,
  name: 'Thor',
  age: 1500,
  salary: 90000,
  city: 'Asgard',
  department: 'Operations'
}
{
```

Limit — first 5 documents

```
> db.employees.find({city:"Lucknow"})  
<
```

Search by city — Lucknow

```
> db.employees.countDocuments()  
< 38
```

AvengersDB >

Count Documents — result: 38

6. MONGODB OPERATORS USED

Operator	Meaning	Purpose
\$eq	Equal To	Checks whether a field equals a specified value.
\$gt	Greater Than	Checks whether a field is greater than a specified value.
\$lt	Less Than	Checks whether a field is less than a specified value.
\$gte	Greater Than or Equal	Checks whether a field is greater than or equal to a specified value.
\$lte	Less Than or Equal	Checks whether a field is less than or equal to a specified value.
\$and	Logical AND	All specified conditions must be satisfied.
\$or	Logical OR	At least one specified condition must be satisfied.
\$set	Update Operator	Changes the value of a field in a document.

7. RESULT / OBSERVATION

The MongoDB operations were executed successfully during the project work. The collection initially contained 50 employee documents. After the delete operations, the final count displayed by `countDocuments()` was 38 documents.

Sorting demonstrated ascending and descending salary order. The limit operation displayed five documents, and the search operation was executed for the city Lucknow.

8. CONCLUSION

The Employee Management System project was successfully developed using MongoDB. The project demonstrated how a NoSQL database can be used to store, retrieve, update, and delete employee information efficiently.

Various MongoDB operations and operators such as `$eq`, `$gt`, `$lt`, `$gte`, `$lte`, `$and`, `$or`, sorting, limiting, updating, deleting, searching and counting documents were implemented.

Through this project, I gained practical knowledge of MongoDB database management and learned how different queries can be used to manipulate and retrieve data from a collection.

Overall, this project helped me understand the basic concepts and practical applications of MongoDB and NoSQL databases.

9. VIVA QUICK REVISION

Question	Answer
What is MongoDB?	MongoDB is a document-oriented NoSQL database.

What is a collection?	A collection stores related MongoDB documents.
What does \$gt mean?	Greater than.
What does \$lt mean?	Less than.
What does \$gte mean?	Greater than or equal to.
What does \$lte mean?	Less than or equal to.
What does \$eq mean?	Equal to.
What does \$and do?	Requires all specified conditions to be true.
What does \$or do?	Matches documents satisfying at least one condition.
What does sort({salary:1}) do?	Sorts salary in ascending order.
What does sort({salary:-1}) do?	Sorts salary in descending order.
What does limit(5) do?	Returns only five documents.
What does countDocuments() do?	Counts documents in the collection.